

Multi-currency accounts, wealth and transfers

One workspace, many currencies. Every account keeps its own money; every total says what it is worth, at which rate, and what it had to leave out.

ProfitSync · 10 September 2026 · working copy `C:\Dev\ProfitSync-Codex` (branch `dev`, uncommitted)

Verified on `npm run dev` · full pre-commit gate and production build green · 925 unit tests across 78 files

1. What a user gets

A person with money in Italy and India adds a euro account and a rupee account to the same workspace.

Nothing else about the app changes: the euro account is in euros, the rupee account is in rupees, and the totals are in whichever currency they chose to read in.

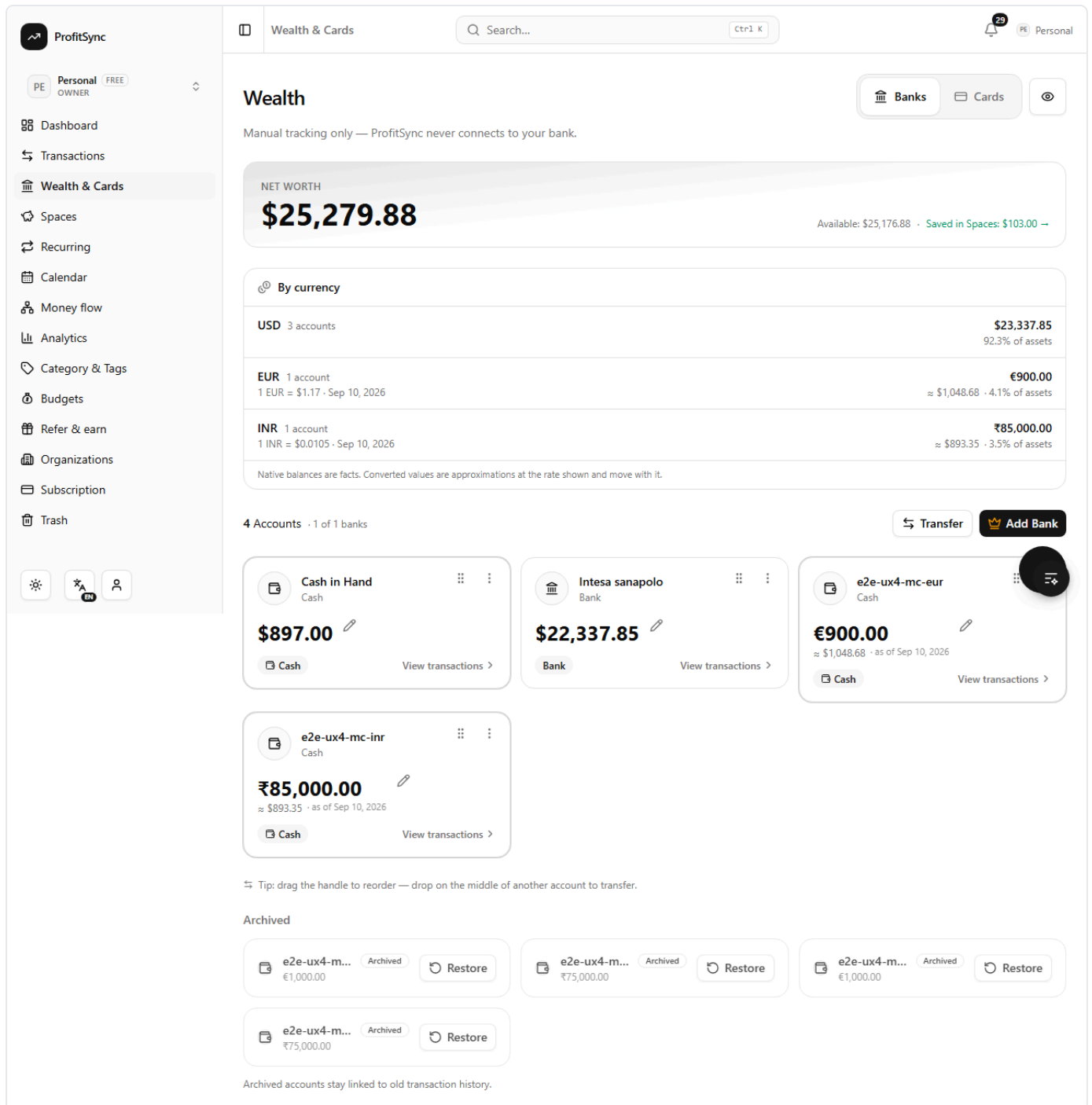


Figure 1 — Wealth. Net worth is the consolidated figure. The *By currency* card gives each currency's native total, its approximate value, the rate used and the date, and its share of assets. Every account tile shows its native balance, with $\approx$ underneath only when it differs from the reporting currency. Rupee amounts are grouped the Indian way.

The one rule everything follows

A native balance is a **fact**. ₹75,000 stays ₹75,000 tomorrow. A converted value is an **approximation** at a stated rate on a stated date, and the screen says so in words. If a currency has no rate, it is listed natively and **excluded** from the total, with the exclusion named. Nothing is ever silently counted as 1:1.

2. Adding money in a foreign account

The transaction form follows the account you pick. Choose the rupee wallet and the amount field asks for rupees; every account in the list shows its own balance in its own currency.

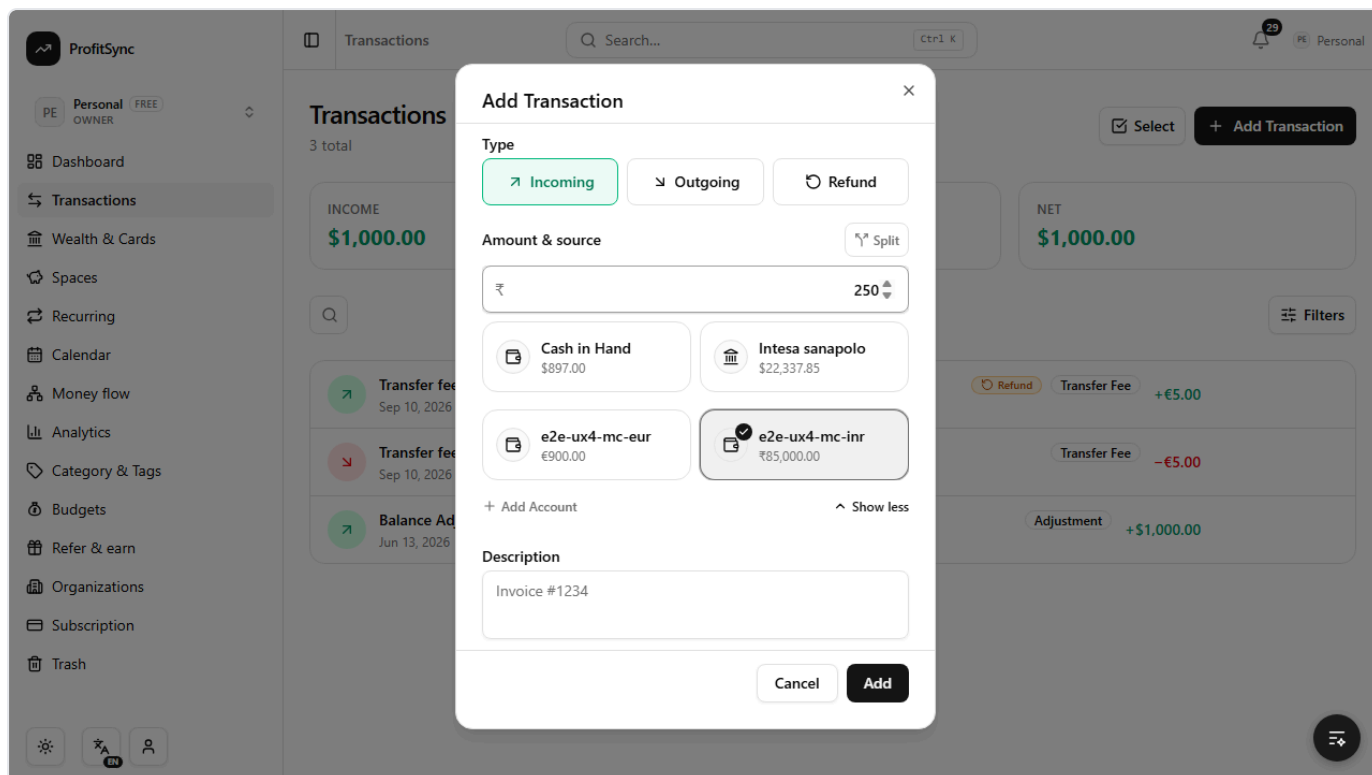


Figure 2 — Add Transaction (personal workspace). The amount carries ₹ because the rupee wallet is selected. Cash in Hand reads \$897.00, the euro wallet €900.00, the rupee wallet ₹85,000.00. The transaction is stored in the account's currency, and the ledger keeps that currency forever.

A split stays in one currency

A split is one purchase paid from several accounts, so it is one amount — and amounts in different currencies cannot be added. Picking a second account in another currency is refused with *“Same currency as USD only”* rather than producing a meaningless total.

Add Transaction

Type

➤ Incoming

➤ Outgoing

↺ Refund

Amount & source

Split

Cash in Hand

\$897.00

\$0.00

Intesa sanapolo

\$22,337.85

+

e2e-ux4-mc-eur

€900.00

Same currency as USD only

+

e2e-ux4-mc-inr

₹85,000.00

Same currency as USD only

+

Split across 1 accounts

\$0.00

Cancel

Add

Figure 3 — Split, guarded. Cash in Hand (USD) is chosen, so both foreign wallets dim and say “Same currency as USD only” — the rule explains itself on the tile instead of failing at save.

3. Moving money between currencies

The transfer form asks what leaves and what arrives. It fills the arrival in for you at today's market rate, and then gets out of the way: banks rarely give the market rate, so the moment you edit the figure it is yours, and what the ledger stores is what you actually received.

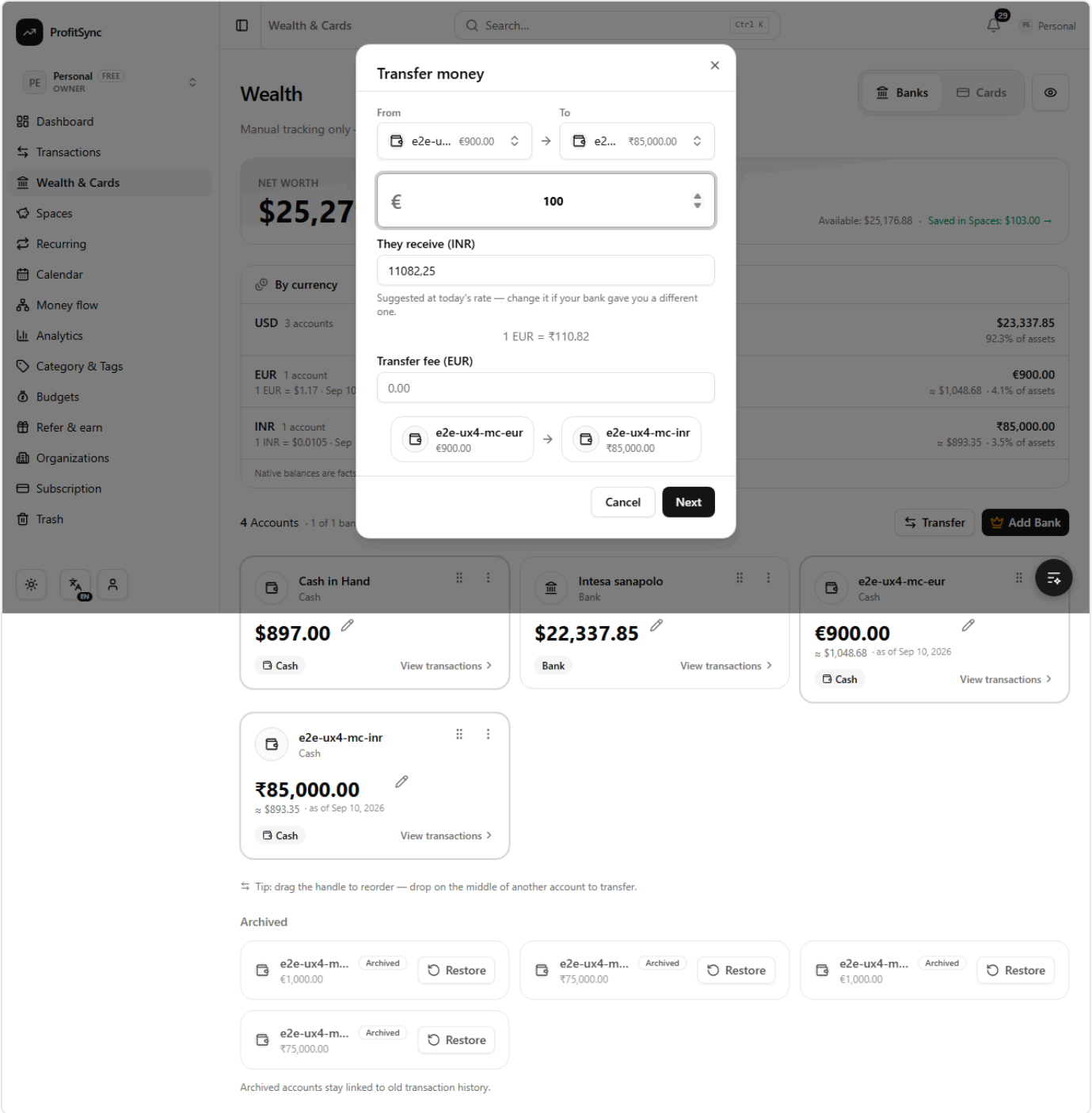


Figure 4 — Cross-currency transfer. €100 proposes ₹11,082.25 at 1 EUR = ₹110.82, labelled “Suggested at today's rate — change it if your bank gave you a different one.” The fee field is in the source currency.

What a transfer does to the books

Event	Source	Destination	Income	Expense	Net worth
-------	--------	-------------	--------	---------	-----------

€300 → €300, same currency	−€300	+€300	0	0	unchanged
€500 → ₹51,350, no fee	−€500	+₹51,350	0	0	≈ unchanged
€500 → ₹51,350, €5 fee	−€505	+₹51,350	0	€5	−€5
Reversal of the above	+€505	−₹51,350	0	fee refunded	restored
Scheduled, not yet done	—	—	0	0	unchanged

The principal is never spending. Only the fee is. Both native amounts and the rate actually obtained are stored on the transfer and never recalculated from a later rate.

4. Reports

Every report converts **each row at its own date**, not at today's rate, so a chart of last year does not move when the rate moves this morning. Each response names the currency it is in and how many rows it could not convert; the screen says so in one calm line rather than under-reporting in silence.

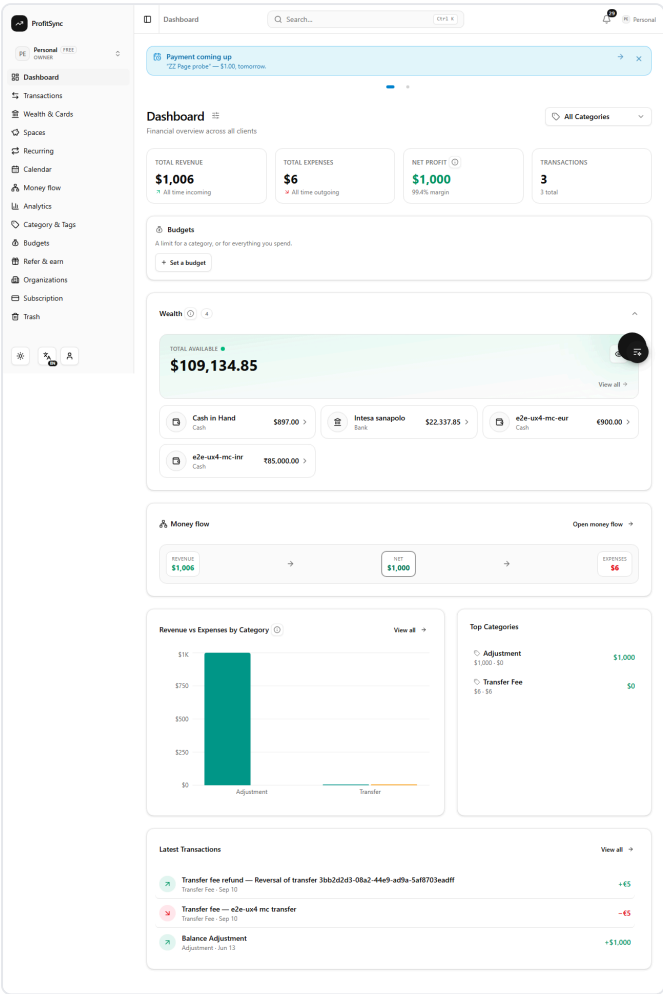


Figure 5 — Dashboard.

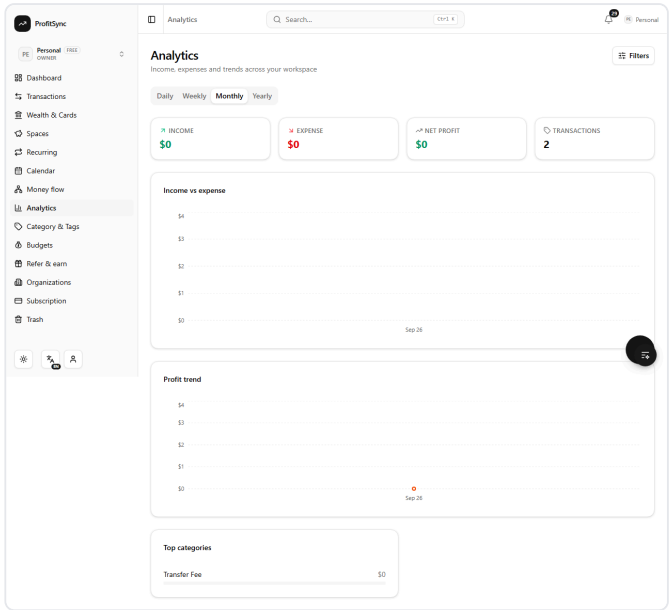


Figure 6 — Analytics.

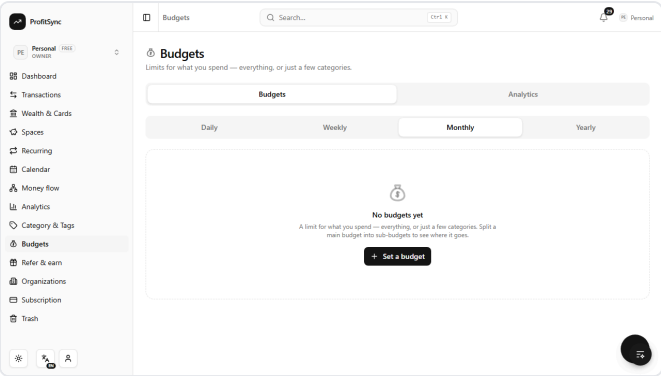


Figure 7 — Budgets.

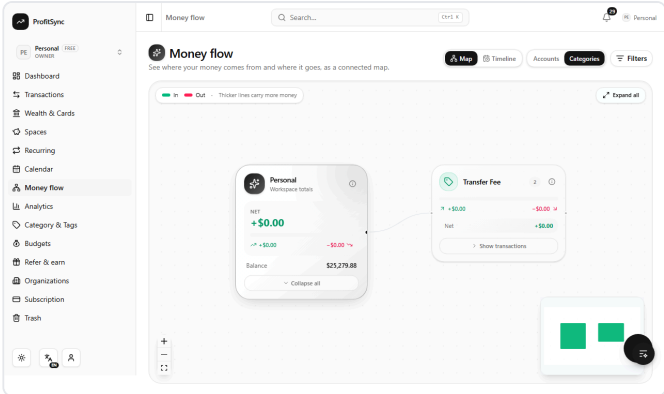


Figure 8 — Money flow.

5. How it is built

Account currency	<code>wealth_accounts.currency_code</code> — set at creation, locked once the account has history
Row currency	<code>transactions.currency_code</code> — snapshot from the account on every write
Reporting currency	<code>organizations.reporting_currency</code> ; <code>currency</code> stays as the compatibility alias
Rates	<code>fx_rate_snapshots</code> — one row per pair per day, with provider and source; weekends carried forward and flagged
Providers	Frankfurter (European Central Bank, has history) then open.er-api for pairs the ECB does not publish, such as AED and SAR
Conversion	SQL functions <code>fx_rate_on()</code> and <code>reporting_amount()</code> (migration 0074), so every aggregate converts per row without a join
Transfers	<code>transfers</code> — one logical row above the two legs, holding both native amounts, the effective rate and the fee
Arithmetic	<code>decimal.js</code> throughout the money paths; no binary floating point in the ledger

New API

Route	What it does
<code>GET /api/wealth/summary</code>	Native totals per currency, converted equivalents, net worth, rate dates, <code>complete</code> and <code>excluded_currencies</code>
<code>GET /api/fx/rate?from=&to=</code>	Today's market rate for one pair; 404 when none exists, so the form just asks for both amounts
<code>GET /api/wealth/transfers</code>	Planned and pending transfers (and completed on request)
<code>PATCH /api/wealth/transfers/:id</code>	planned → pending → completed, or cancelled
<code>POST /api/wealth/transfers/:id/reverse</code>	Reverses a completed transfer once, using its own stored amounts

Privacy

A rate provider receives a currency pair and a date. It never receives an amount, an account name or a description. Conversion happens inside ProfitSync or in the database.

6. What Maqbool needs to do

1. **Nothing is committed.** The work sits in `C:\Dev\ProfitSync-Codex` . `C:\Dev\ProfitSync` is untouched.
2. **Migrations 0069–0074 are already applied to the shared dev database.** Do not re-run them. done

3. **Check the migration journal before merging anything else.** These entries carry `when` values from `1788980400000` up. A branch whose migration has a lower value will be *silently skipped* on a database that already ran these — the Debt & Loans branch is the one to check. action
4. **Native parity has not been run** — `npm run cap:sync:android` and `npm run cap:sync:ios`. action
5. **Suggested commit split**, in order: currency and FX foundations; native currency on every financial write; cross-currency transfers, fees and lifecycle; conversion-aware reports; multi-currency UI and localisation; docs and end-to-end coverage.

Two behaviours worth a decision

A reversed transfer cannot be trashed. By design: trashing one half of a reversal pair would leave balances wrong. The consequence is that an account which took part in one can only be archived, never deleted. Reasonable for an audit trail, but it is a product choice, not an accident.

Extra cash wallets used to be undeletable. The old guard refused to delete *any* cash account, which was correct when a workspace had exactly one. Now that a workspace can hold one wallet per currency, only the auto-provisioned “Cash in Hand” is permanent. Fixed here.

7. Verification

Check	Result
Type check, lint, secret scan, route guards	pass
Unit tests	925 passed across 78 files
Translations, cache map, ESM extensions, function boot	pass
Production build	pass
<code>e2e/multi-currency.spec.ts</code> against localhost	7 passed
Personal-workspace walkthrough	5 passed
Existing smoke, alerts and budgets suites	pass, except one pre-existing failure below

`e2e/smoke.spec.ts` “create a client” fails because its locator finds the hidden floating-action-button item named “Add Client” before the visible “New Client” button. It fails identically on the untouched original repository, so it predates this work.

What the money tests pin

- An account's native balance does not move when a rate moves.
- A cross-currency transfer keeps both native amounts and its own effective rate.
- The fee is the only expense; the principal is never spending; a transfer is never income.
- Reversal restores both sides exactly once and refuses a second time.
- A planned or cancelled transfer moves no money; completing one moves it exactly once.
- Net worth is the sum of converted balances, never of raw numbers across currencies.

8. Not built yet

- **Destination-side fees.** Only a source-side fee is modelled.
- **Historical net-worth chart.** The per-day rate snapshots exist; no chart values balances at each point's own date yet.
- **Currency-movement attribution** — separating “I saved” from “the rate moved”. Designed for, not built.
- **Cross-currency recurring transfers** are refused with a clear message until they carry an explicit rate policy.
- **A foreign key on `clients.organization_id`**, which has none; the dev database holds ten orphaned client rows. Unrelated to this feature.

Companion documents: `docs/multi-currency/HANDOFF_PROMPT.md` (paste into a fresh Claude Code session), `docs/multi-currency/ARCHITECTURE.md` (design and invariants), `MULTI_CURRENCY_IMPLEMENTATION_STATUS.md` (per-writer audit).